

## ЛР 6

SQL-код триггеров

```
-- 1

-- При добавлении нового топпинга к строке заказа проверяет склад и уменьшает
остаток нужного ингредиента

CREATE OR REPLACE FUNCTION trg_decrease_stock_on_topping_insert()
RETURNS trigger
LANGUAGE plpgsql
AS $$
DECLARE
    v_bakery_id bigint;
    v_price_list_item_id bigint;
    v_quantity integer;
    v_allowed_ingredient_id bigint;
    v_required_g numeric(12,3);
    v_required_pcs integer;
    v_stock_g numeric(12,3);
    v_stock_pcs integer;
BEGIN
    -- Выбираем заказ в который будем вставлять топинг и сразу достаём
кондитерскую
    SELECT co.bakery_id, oi.price_list_item_id, oi.quantity
    INTO v_bakery_id, v_price_list_item_id, v_quantity
    FROM order_item oi
    JOIN customer_order co ON co.order_id = oi.order_id
    WHERE oi.order_item_id = NEW.order_item_id;

    IF NOT FOUND THEN
        RAISE EXCEPTION 'Строка заказа % не найдена', NEW.order_item_id;
    END IF;

    -- Проверяем можно ли такой топинг вообще добавить
    SELECT atp.ingredient_id
    INTO v_allowed_ingredient_id
    FROM allowed_topping atp
    WHERE atp.price_list_item_id = v_price_list_item_id
        AND atp.ingredient_id = NEW.ingredient_id
        AND atp.amount_unit = NEW.amount_unit;

    IF NOT FOUND THEN
        RAISE EXCEPTION 'Топпинг % не разрешён для позиции прайса % или указана
неверная единица измерения',
            NEW.ingredient_id, v_price_list_item_id;
    END IF;

    -- Может быть в граммах или в штуках, смотрим хватает ли на складе и если да
уменьшаем, иначе падаем с ошибкой
```

```

IF NEW.amount_unit = 'g' THEN
    v_required_g := NEW.amount_value * v_quantity;

    SELECT stock_qty_g
    INTO v_stock_g
    FROM bakery_ingredient_stock
    WHERE bakery_id = v_bakery_id
        AND ingredient_id = NEW.ingredient_id
    FOR UPDATE;

    IF NOT FOUND THEN
        RAISE EXCEPTION 'Топпинг % отсутствует на складе кондитерской %',
            NEW.ingredient_id, v_bakery_id;
    END IF;

    IF v_stock_g < v_required_g THEN
        RAISE EXCEPTION 'Недостаточно топпинга % на складе кондитерской %.
        Нужно % г, доступно % г',
            NEW.ingredient_id, v_bakery_id, v_required_g, v_stock_g;
    END IF;

    UPDATE bakery_ingredient_stock
    SET stock_qty_g = stock_qty_g - v_required_g
    WHERE bakery_id = v_bakery_id
        AND ingredient_id = NEW.ingredient_id;

ELSIF NEW.amount_unit = 'pcs' THEN
    v_required_pcs := (NEW.amount_value * v_quantity)::integer;

    SELECT stock_qty_pcs
    INTO v_stock_pcs
    FROM bakery_ingredient_stock
    WHERE bakery_id = v_bakery_id
        AND ingredient_id = NEW.ingredient_id
    FOR UPDATE;

    IF NOT FOUND THEN
        RAISE EXCEPTION 'Топпинг % отсутствует на складе кондитерской %',
            NEW.ingredient_id, v_bakery_id;
    END IF;

    IF v_stock_pcs < v_required_pcs THEN
        RAISE EXCEPTION 'Недостаточно топпинга % на складе кондитерской %.
        Нужно % шт., доступно % шт.',
            NEW.ingredient_id, v_bakery_id, v_required_pcs, v_stock_pcs;
    END IF;

    UPDATE bakery_ingredient_stock
    SET stock_qty_pcs = stock_qty_pcs - v_required_pcs
    WHERE bakery_id = v_bakery_id
        AND ingredient_id = NEW.ingredient_id;

```

```

        END IF;

        RETURN NEW;
    END;
$$;

CREATE TRIGGER trg_decrease_stock_on_topping_insert
BEFORE INSERT ON order_item_topping
FOR EACH ROW
EXECUTE FUNCTION trg_decrease_stock_on_topping_insert();

-- 2
-- При добавления графика работы проверять не пересекается ли он с предыдущим
текущим графиком работы работника и не выходит за рамки рабочих часов
кондитерской больше чем на час
-- В случае если выходит, вывести ошибку и данные не добавлять

CREATE OR REPLACE FUNCTION trg_validate_work_schedule()
RETURNS trigger
LANGUAGE plpgsql
AS $$
DECLARE
    v_open_time time;
    v_close_time time;
    v_conflict_schedule_id bigint;
BEGIN
    -- Получаем рабочие часы кондитерской для которой добавляем график работы
    SELECT open_time, close_time
    INTO v_open_time, v_close_time
    FROM bakery
    WHERE bakery_id = NEW.bakery_id;

    IF NOT FOUND THEN
        RAISE EXCEPTION 'Кондитерская % не найдена', NEW.bakery_id;
    END IF;

    -- Проверяем что мы не выходим за пределы часа
    IF NEW.work_start_time < (v_open_time - INTERVAL '1 hour')::time
        OR NEW.work_end_time > (v_close_time + INTERVAL '1 hour')::time THEN
        RAISE EXCEPTION 'График сотрудника % на дату % выходит за допустимые
рамки кондитерской %',
            NEW.employee_id, NEW.work_date, NEW.bakery_id;
    END IF;

    -- Смотрим отсутствие пересечения с каким-то другим графиком сотрудника
    SELECT ws.schedule_id
    INTO v_conflict_schedule_id
    FROM work_schedule ws
    WHERE ws.employee_id = NEW.employee_id
        AND ws.work_date = NEW.work_date
        AND ws.schedule_id <> COALESCE(NEW.schedule_id, -1)

```

```

        AND NEW.work_start_time < ws.work_end_time
        AND NEW.work_end_time > ws.work_start_time;

    IF FOUND THEN
        RAISE EXCEPTION 'График сотрудника % на дату % пересекается с графиком
%',
            NEW.employee_id, NEW.work_date, v_conflict_schedule_id;
    END IF;

    RETURN NEW;
END;
$$;

CREATE TRIGGER trg_validate_work_schedule
BEFORE INSERT OR UPDATE ON work_schedule
FOR EACH ROW
EXECUTE FUNCTION trg_validate_work_schedule();

```

SQL-код процедур

```

-- 1
-- Расчёт итоговой стоимости заказа с учётом всех топпингов
-- На вход принимает номер заказа
-- Если заказа нет, формируется ошибка
-- После успешного пересчёта обновляются строки заказа и сам заказ

CREATE OR REPLACE PROCEDURE proc_recalculate_order_total(
    IN p_order_id bigint
)
LANGUAGE plpgsql
AS $$
DECLARE
    v_order_id bigint;
    v_total numeric(12,2);
    v_item record;
    v_toppings_price numeric(12,2);
    v_taste_drop_pct numeric(5,2);
BEGIN
    -- Проверяем, существует ли такой заказ
    SELECT order_id
    INTO v_order_id
    FROM customer_order
    WHERE order_id = p_order_id;

    IF NOT FOUND THEN
        RAISE EXCEPTION 'Заказ % не найден', p_order_id;
    END IF;

    -- Пересчитываем каждую строку заказа отдельно

```

```

FOR v_item IN
    SELECT
        oi.order_item_id,
        oi.quantity,
        pli.sale_price
    FROM order_item oi
    JOIN price_list_item pli ON pli.price_list_item_id =
oi.price_list_item_id
    WHERE oi.order_id = p_order_id
LOOP
    -- Считаем стоимость топпингов и падение вкуса для конкретной строки
заказа
    SELECT
        COALESCE(SUM(extra_price), 0),
        COALESCE(SUM(taste_drop_pct_applied), 0)
    INTO v_toppings_price, v_taste_drop_pct
    FROM order_item_topping
    WHERE order_item_id = v_item.order_item_id;

    -- Обновляем строку заказа
    UPDATE order_item
    SET
        base_price = v_item.sale_price,
        toppings_price = v_toppings_price,
        line_total_amount = (v_item.sale_price + v_toppings_price) *
v_item.quantity,
        final_taste_pct = GREATEST(100 - v_taste_drop_pct, 0)
    WHERE order_item_id = v_item.order_item_id;
END LOOP;

-- Считаем итоговую сумму заказа
SELECT COALESCE(SUM(line_total_amount), 0)
INTO v_total
FROM order_item
WHERE order_id = p_order_id;

-- Обновляем сам заказ
UPDATE customer_order
SET order_total_amount = v_total
WHERE order_id = p_order_id;

RAISE NOTICE 'Стоимость заказа % пересчитана: %', p_order_id, v_total;
END;
$$;

-- 2
-- Оформление заказа
-- Если p_order_id = NULL, создаётся новый заказ и новая строка заказа
-- Если p_order_id задан, строка добавляется в существующий заказ
-- Если такая продукция уже есть в заказе, увеличивается количество
-- p_result_order_id – OUT-параметр, в него возвращается номер заказа

```

```

CREATE OR REPLACE PROCEDURE proc_make_order(
    IN p_order_id bigint,
    IN p_client_id bigint,
    IN p_bakery_id bigint,
    IN p_cashier_id bigint,
    IN p_order_type text,
    IN p_status text,
    IN p_price_list_item_id bigint,
    IN p_quantity integer,
    OUT p_result_order_id bigint
)
LANGUAGE plpgsql
AS $$
DECLARE
    v_item_bakery_id bigint;
    v_order_bakery_id bigint;
    v_base_price numeric(12,2);
    v_changed_count int;
BEGIN
    IF p_quantity IS NULL OR p_quantity <= 0 THEN
        RAISE EXCEPTION 'Количество продукции должно быть положительным';
    END IF;

    -- Проверяем что позиция прайса вообще существует (такое вообще можно
    заказать)
    SELECT pl.bakery_id, pli.sale_price
    INTO v_item_bakery_id, v_base_price
    FROM price_list_item pli
    JOIN price_list pl ON pl.price_list_id = pli.price_list_id
    WHERE pli.price_list_item_id = p_price_list_item_id
        AND pl.is_active = TRUE;

    IF NOT FOUND THEN
        RAISE EXCEPTION 'Позиция прайса % не найдена или прайс не активен',
p_price_list_item_id;
    END IF;

    -- Создаем или новый заказ или дописываем в старый
    IF p_order_id IS NULL THEN
        IF p_client_id IS NULL OR p_bakery_id IS NULL OR p_cashier_id IS NULL
            OR p_order_type IS NULL OR p_status IS NULL THEN
            RAISE EXCEPTION 'Для нового заказа нужно передать client_id,
bakery_id, cashier_id, order_type и status';
        END IF;

        IF p_bakery_id <> v_item_bakery_id THEN
            RAISE EXCEPTION 'Позиция прайса % относится к кондитерской %, а заказ
создаётся для кондитерской %',
                p_price_list_item_id, v_item_bakery_id, p_bakery_id;
        END IF;

```

```

INSERT INTO customer_order (
    client_id, bakery_id, cashier_id,
    order_type, status, order_total_amount
) VALUES (
    p_client_id, p_bakery_id, p_cashier_id,
    p_order_type, p_status, 0
);

SELECT MAX(order_id)
INTO p_result_order_id
FROM customer_order
WHERE client_id = p_client_id
    AND bakery_id = p_bakery_id
    AND cashier_id = p_cashier_id
    AND order_type = p_order_type
    AND status = p_status;
ELSE
SELECT bakery_id
INTO v_order_bakery_id
FROM customer_order
WHERE order_id = p_order_id;

IF NOT FOUND THEN
    RAISE EXCEPTION 'Заказ % не найден', p_order_id;
END IF;

IF v_order_bakery_id <> v_item_bakery_id THEN
    RAISE EXCEPTION 'Позиция прайса % относится к кондитерской %, а заказ
% относится к кондитерской %',
        p_price_list_item_id, v_item_bakery_id, p_order_id,
v_order_bakery_id;
END IF;

p_result_order_id := p_order_id;
END IF;

UPDATE order_item
SET quantity = quantity + p_quantity
WHERE order_id = p_result_order_id
    AND price_list_item_id = p_price_list_item_id;

GET DIAGNOSTICS v_changed_count = ROW_COUNT;

IF v_changed_count = 0 THEN
    INSERT INTO order_item (
        order_id, price_list_item_id, quantity,
        base_price, toppings_price, line_total_amount, final_taste_pct
    ) VALUES (
        p_result_order_id, p_price_list_item_id, p_quantity,
        v_base_price, 0, v_base_price * p_quantity, 100

```

```

    );
END IF;

CALL proc_recalculate_order_total(p_result_order_id);

RAISE NOTICE 'Заказ % оформлен или обновлён', p_result_order_id;
END;
$$;

```

#### Тестовые сценарии:

```

-- 1 ПОЗИТИВНЫЙ ТЕСТ

-- топпинг добавляется, остаток малины уменьшается с 1000 до 976 г

DELETE FROM customer_order
WHERE order_type = 'TEST_TRIGGER_RUN'
    AND status = 'TOPPING_OK';

DELETE FROM bakery_ingredient_stock
WHERE bakery_id = 1 AND ingredient_id = 8;

INSERT INTO bakery_ingredient_stock (
    bakery_id, ingredient_id, stock_qty_g, stock_qty_pcs,
    avg_daily_consumption_g, avg_daily_consumption_pcs
) VALUES (
    1, 8, 1000.000, 0, 0.000, 0
);

INSERT INTO customer_order (
    client_id, bakery_id, cashier_id,
    order_type, status, order_total_amount
) VALUES (
    1, 1, 4, 'TEST_TRIGGER_RUN', 'TOPPING_OK', 0
);

INSERT INTO order_item (
    order_id, price_list_item_id, quantity,
    base_price, toppings_price, line_total_amount, final_taste_pct
) VALUES (
    (SELECT MAX(order_id)
     FROM customer_order
     WHERE order_type = 'TEST_TRIGGER_RUN'
        AND status = 'TOPPING_OK'),
    1, 2, 320.00, 0.00, 640.00, 100.00
);

INSERT INTO order_item_topping (
    order_item_id, ingredient_id, amount_value, amount_unit,
    extra_price, taste_drop_pct_applied

```



```

) VALUES (
  (SELECT MAX(oi.order_item_id)
   FROM order_item oi
   JOIN customer_order co ON co.order_id = oi.order_id
   WHERE co.order_type = 'TEST_TRIGGER_RUN'
        AND co.status = 'TOPPING_OK'),
  8, 12.000, 'g', 70.00, 6.00
);

SELECT 'Ожидалось 976.000 г' AS check_name, stock_qty_g AS actual_stock_g
FROM bakery_ingredient_stock
WHERE bakery_id = 1 AND ingredient_id = 8;

```

```

-- 1 негативный тест
-- последний INSERT завершается ошибкой, потому что ингредиент 1 не разрешён как
топпинг для позиции прайса 1

DELETE FROM customer_order
WHERE order_type = 'TEST_TRIGGER_RUN'
      AND status = 'TOPPING_FORBIDDEN';

INSERT INTO customer_order (
  client_id, bakery_id, cashier_id,
  order_type, status, order_total_amount
) VALUES (
  1, 1, 4, 'TEST_TRIGGER_RUN', 'TOPPING_FORBIDDEN', 0
);

INSERT INTO order_item (
  order_id, price_list_item_id, quantity,
  base_price, toppings_price, line_total_amount, final_taste_pct
) VALUES (
  (SELECT MAX(order_id)
   FROM customer_order
   WHERE order_type = 'TEST_TRIGGER_RUN'
        AND status = 'TOPPING_FORBIDDEN'),
  1, 1, 320.00, 0.00, 320.00, 100.00
);

INSERT INTO order_item_topping (
  order_item_id, ingredient_id, amount_value, amount_unit,
  extra_price, taste_drop_pct_applied
) VALUES (
  (SELECT MAX(oi.order_item_id)
   FROM order_item oi
   JOIN customer_order co ON co.order_id = oi.order_id
   WHERE co.order_type = 'TEST_TRIGGER_RUN'
        AND co.status = 'TOPPING_FORBIDDEN'),
  1, 10.000, 'g', 10.00, 1.00
);

```

```
);
```

```
-- 2 позитивный тест  
-- две непересекающиеся смены одного сотрудника в один день добавляются
```

```
DELETE FROM work_schedule  
WHERE employee_id = 3  
AND work_date = DATE '2026-06-01';
```

```
INSERT INTO work_schedule (  
    employee_id, bakery_id, work_date, day_of_week,  
    work_start_time, work_end_time, break_start_time, break_end_time  
) VALUES (  
    3, 1, DATE '2026-06-01', 1,  
    '07:00', '10:00', NULL, NULL  
);
```

```
INSERT INTO work_schedule (  
    employee_id, bakery_id, work_date, day_of_week,  
    work_start_time, work_end_time, break_start_time, break_end_time  
) VALUES (  
    3, 1, DATE '2026-06-01', 1,  
    '10:00', '12:00', NULL, NULL  
);
```

```
SELECT 'Ожидалось 2 графика' AS check_name, COUNT(*) AS actual_count  
FROM work_schedule  
WHERE employee_id = 3  
AND work_date = DATE '2026-06-01';
```

```
-- 2 негативный тест  
-- INSERT завершается ошибкой, потому что смена начинается раньше допустимой  
рамки  
-- Кондитерская 1 работает с 08:00, допустимое начало не раньше 07:00
```

```
DELETE FROM work_schedule  
WHERE employee_id = 3  
AND work_date = DATE '2026-06-03';
```

```
INSERT INTO work_schedule (  
    employee_id, bakery_id, work_date, day_of_week,  
    work_start_time, work_end_time, break_start_time, break_end_time  
) VALUES (  
    3, 1, DATE '2026-06-03', 3,  
    '06:30', '09:00', NULL, NULL  
);
```

```

-- Процедура 1, позитивный тест.
-- Ожидаемый результат: заказ 1 пересчитывается обратно к сумме 380.00.

UPDATE order_item
SET base_price = 1,
    toppings_price = 1,
    line_total_amount = 1,
    final_taste_pct = 1
WHERE order_item_id = 1;

UPDATE customer_order
SET order_total_amount = 1
WHERE order_id = 1;

CALL proc_recalculate_order_total(1);

SELECT 'Строка заказа: ожидалось 380.00' AS check_name, line_total_amount AS
actual_value
FROM order_item
WHERE order_item_id = 1;

SELECT 'Заказ: ожидалось 380.00' AS check_name, order_total_amount AS
actual_value
FROM customer_order
WHERE order_id = 1;

```

```

-- 1 негативный тест
-- CALL завершается ошибкой, потому что заказа 999999 нет

CALL proc_recalculate_order_total(999999);

```

```

-- 2 позитивный тест
-- создаётся новый заказ и одна строка заказа.

DELETE FROM customer_order
WHERE order_type = 'TEST_PROC_RUN'
    AND status = 'NEW_ORDER';

CALL proc_make_order(
    NULL,
    1,
    1,

```

```

4,
'TEST_PROC_RUN',
'NEW_ORDER',
1,
2,
NULL
);

SELECT 'Ожидался 1 заказ' AS check_name, COUNT(*) AS actual_orders
FROM customer_order
WHERE order_type = 'TEST_PROC_RUN'
AND status = 'NEW_ORDER';

SELECT 'Ожидалась 1 строка заказа с количеством 2' AS check_name,
COUNT(*) AS actual_items,
SUM(quantity) AS actual_quantity
FROM order_item oi
JOIN customer_order co ON co.order_id = oi.order_id
WHERE co.order_type = 'TEST_PROC_RUN'
AND co.status = 'NEW_ORDER'
AND oi.price_list_item_id = 1;

```

```

-- 2 негативный тест
-- CALL завершается ошибкой, потому что заказа 999999 нет

CALL proc_make_order(
999999,
NULL,
NULL,
NULL,
NULL,
NULL,
1,
1,
NULL
);

```

Примеры выполнения (производилось после test\_insert для удобства):

Query

Query History

1

-- 1 позитивный тест

2

-- топпинг добавляется, остаток малины уменьшается с 1000 до 976 г

3

4

DELETE FROM customer\_order

5

WHERE order\_type = 'TEST\_TRIGGER\_RUN'

6

AND status = 'TOPPING\_OK';

7

8

DELETE FROM bakery\_ingredient\_stock

9

WHERE bakery\_id = 1 AND ingredient\_id = 8;

10

11

INSERT INTO bakery\_ingredient\_stock (

12

bakery\_id, ingredient\_id, stock\_qty\_g, stock\_qty\_pcs,

13

avg\_daily\_consumption\_g, avg\_daily\_consumption\_pcs

14

) VALUES (

15

1, 8, 1000.000, 0, 0.000, 0

16

);

17

18

INSERT INTO customer\_order (

19

client\_id, bakery\_id, cashier\_id,

20

order\_type, status, order\_total\_amount

Data Output

Messages

Notifications

≡

📄

▼

📋

▼

🗑

🗑

📄

⬇

📈

SQL

|   | check_name<br>text  | actual_stock_g<br>numeric (12,3) |
|---|---------------------|----------------------------------|
| 1 | Ожидалось 976.000 г | 976.000                          |

1

-- 1 негативный тест

2

-- последний INSERT завершается ошибкой, потому что ингредиент 1 не разрешён как топпинг для позиции прайса 1

3

4

DELETE FROM customer\_order

5

WHERE order\_type = 'TEST\_TRIGGER\_RUN'

6

AND status = 'TOPPING\_FORBIDDEN';

7

8

INSERT INTO customer\_order (

9

client\_id, bakery\_id, cashier\_id,

10

order\_type, status, order\_total\_amount

11

) VALUES (

12

1, 1, 4, 'TEST\_TRIGGER\_RUN', 'TOPPING\_FORBIDDEN', 0

13

);

14

15

INSERT INTO order\_item (

16

order\_id, price\_list\_item\_id, quantity,

17

base\_price, toppings\_price, line\_total\_amount, final\_taste\_pct

18

) VALUES (

19

(SELECT MAX(order\_id)

20

FROM customer\_order

Data Output

Messages

Notifications

ERROR: Топпинг 1 не разрешён для позиции прайса 1 или указана неверная единица измерения

CONTEXT: функция PL/pgSQL trg\_decrease\_stock\_on\_topping\_insert(), строка 32, оператор RAISE

ОШИБКА: Топпинг 1 не разрешён для позиции прайса 1 или указана неверная единица измерения

SQL state: P0001

```

1  -- 2 позитивный тест
2  -- две непересекающиеся смены одного сотрудника в один день добавляются
3
4  DELETE FROM work_schedule
5  WHERE employee_id = 3
6         AND work_date = DATE '2026-06-01';
7
8  INSERT INTO work_schedule (
9      employee_id, bakery_id, work_date, day_of_week,
10     work_start_time, work_end_time, break_start_time, break_end_time
11 ) VALUES (
12     3, 1, DATE '2026-06-01', 1,
13     '07:00', '10:00', NULL, NULL
14 );
15
16 INSERT INTO work_schedule (
17     employee_id, bakery_id, work_date, day_of_week,
18     work_start_time, work_end_time, break_start_time, break_end_time
19 ) VALUES (
20     3, 1, DATE '2026-06-01', 1,

```

Data Output Messages Notifications

|   | check_name          | actual_count |
|---|---------------------|--------------|
|   | text                | bigint       |
| 1 | Ожидалось 2 графика | 2            |

Query Query History

```

1  -- 2 негативный тест
2  -- INSERT завершается ошибкой, потому что смена начинается раньше допустимой рамки
3  -- Кондитерская 1 работает с 08:00, допустимое начало не раньше 07:00
4
5  DELETE FROM work_schedule
6  WHERE employee_id = 3
7         AND work_date = DATE '2026-06-03';
8
9  INSERT INTO work_schedule (
10     employee_id, bakery_id, work_date, day_of_week,
11     work_start_time, work_end_time, break_start_time, break_end_time
12 ) VALUES (
13     3, 1, DATE '2026-06-03', 3,
14     '06:30', '09:00', NULL, NULL
15 );
16

```

Data Output Messages Notifications

ERROR: График сотрудника 3 на дату 2026-06-03 выходит за допустимые рамки кондитерской 1  
CONTEXT: функция PL/pgSQL trg\_validate\_work\_schedule(), строка 20, оператор RAISE

ОШИБКА: График сотрудника 3 на дату 2026-06-03 выходит за допустимые рамки кондитерской 1  
SQL state: P0001

```

1  -- Процедура 1, позитивный тест.
2  -- Ожидаемый результат: заказ 1 пересчитывается обратно к сумме 380.00.
3
4  UPDATE order_item
5  SET base_price = 1,
6      toppings_price = 1,
7      line_total_amount = 1,
8      final_taste_pct = 1
9  WHERE order_item_id = 1;
10
11 UPDATE customer_order
12 SET order_total_amount = 1
13 WHERE order_id = 1;
14
15 CALL proc_recalculate_order_total(1);
16
17 SELECT 'Строка заказа: ожидалось 380.00' AS check_name, line_total_amount AS actual_value
18 FROM order_item
19 WHERE order_item_id = 1;
20

```

Data Output Messages Notifications

|   | check_name<br>text      | actual_value<br>numeric (12,2) |
|---|-------------------------|--------------------------------|
| 1 | Заказ: ожидалось 380.00 | 380.00                         |

```
1  -- 1 негативный тест
2  -- CALL завершается ошибкой, потому что заказа 999999 нет
3
4  CALL proc_recalculate_order_total(999999);
5
```

Data Output Messages Notifications

ERROR: Заказ 999999 не найден

CONTEXT: функция PL/pgSQL proc\_recalculate\_order\_total(bigint), строка 16, оператор RAISE

ОШИБКА: Заказ 999999 не найден

SQL state: P0001



Open in a new tab

history

```
1  -- 2 позитивный тест
2  -- создаётся новый заказ и одна строка заказа.
3
4  DELETE FROM customer_order
5  WHERE order_type = 'TEST_PROC_RUN'
6         AND status = 'NEW_ORDER';
7
8  CALL proc_make_order(
9      NULL,
10     1,
11     1,
12     4,
13     'TEST_PROC_RUN',
14     'NEW_ORDER',
15     1,
16     2,
17     NULL
18 );
19
20 SELECT 'Ожидался 1 заказ' AS check_name, COUNT(*) AS actual_order;
```

Data Output Messages Notifications

|   | check_name<br>text                        | actual_items<br>bigint | actual_quantity<br>bigint |
|---|-------------------------------------------|------------------------|---------------------------|
| 1 | Ожидалась 1 строка заказа с количеством 2 | 1                      | 2                         |

Query Query History

```
1  -- 2 негативный тест
2  -- CALL завершается ошибкой, потому что заказа 999999 нет
3
4  CALL proc_make_order(
5      999999,
6      NULL,
7      NULL,
8      NULL,
9      NULL,
10     NULL,
11     1,
12     1,
13     NULL
14 );
15
```

Data Output Messages Notifications

ERROR: Заказ 999999 не найден  
CONTEXT: функция PL/pgSQL proc\_make\_order(bigint,bigint,bigint,bigint,text,text,bigint,integer), строка 59, оператор RAISE  
  
ОШИБКА: Заказ 999999 не найден  
SQL state: P0001